

Lossless Compressibility of Embeddings (20260113)

Objective: To find the impact of bitplane
dissaggregation on compressibility of vector
embeddings

Data Collection Methodology:

These were kept constant across all runs of data collection:

- # of lines analyzed: 1024
- # of vector embeddings: 1024
- Quantization datatype: INT8
- Scalar datatype: FP32

Here is how the metrics are calculated:

- % Space savings = $(1 - (a/b)) * 100$
- Where:
 - a = new size
 - b = original size
 - Result rounded to the 3rd decimal point

Compression Codecs: ZSTD & LZ4

Unit of Compression: 1 Vector Embedding
Token Chunking: By-line (model max context, splits lines that are too long)

These parameters would change depending on the embedding model used:

- Base Datatype: FP32, FP16, BFloat16
- Block Size: = Dimension * bit size (quantization datatype) / 8 (bit packing)
 - Ex: Dimensions = 1024, Quant dtype=INT8, so Block Size = 1024 (1 KiB)
- Max Chunk Size: Max token context for model
- Dimension: 768, 1024, 2560, etc

Certain embedding models require a query or keyword to make tailored computed vector embeddings for real-world RAG retrieval workloads.

Models and Datasets used + Runtime Constants:

Embedding Models:

- BAAI BGE-M3 (FP16)
- EmbeddingGemma 300M (FP16) [Based on Gemma 3]
- Qwen3 4B (FP16)
- Harrier OSS V1 0.6B (BF16)
- Arctic Snowflake Embed V2.0 (FP32)
- E5 Large V2 (FP32)
- Nomic Embed V1.5 (FP32)

These were kept constant across all runs of data collection:

- # of lines analyzed: 1024
- # of vector embeddings: 1024
- Quantization datatype: INT8
- Scalar datatype: FP32

Datasets:

- Wikipedia DPR (Precomputed)
- Fineweb-Edu (Precomputed)
- Cohere MSMARCO V2.1 English V3 (Based on TREC RAG 2024) (Precomputed)

The 9 Datasets below are averaged for simplicity:

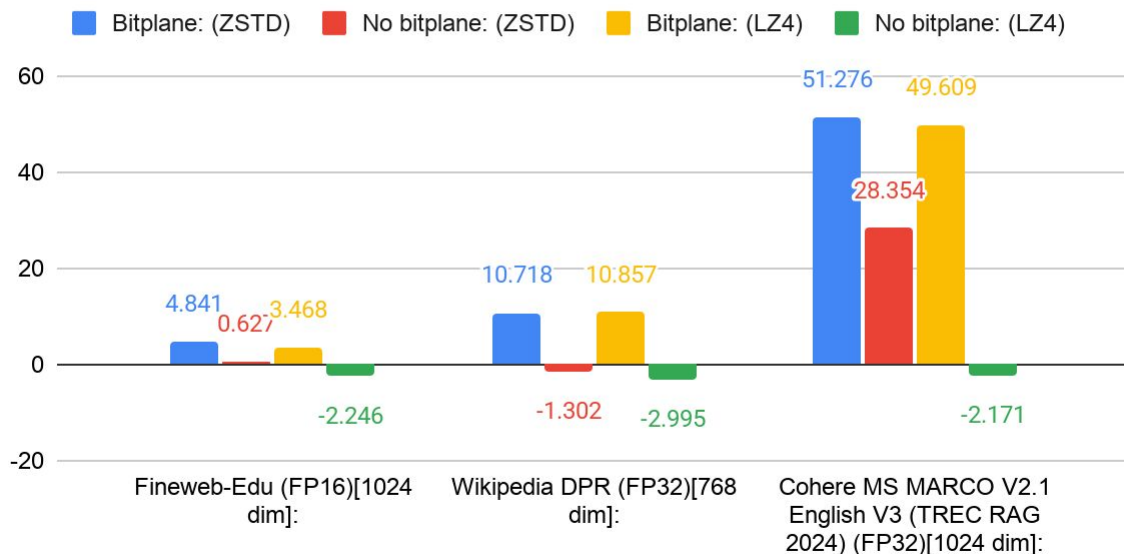
- Booksum (Computed)
- Fineweb-Edu emb (Computed)
- Wikitext-103 (Computed)
- Amazon vector database (Computed)
- BEIR nfcorpus (Computed)
- BEIR fiqa (Computed)
- BEIR touche-2020 (Computed)
- BEIR dbpedia-entity (Computed)
- BEIR fever (Computed)

Precomputed Results: (FP16 & FP32)

For ZSTD and LZ4 compression, bitplaning shows better memory savings than no bitplaning.

MS MARCO has considerably higher compressibility compared to the other two precomputed datasets.

Precomputed Unquantized: % Space Savings Bitplane and No Bitplane vs Baseline (ZSTD & LZ4)



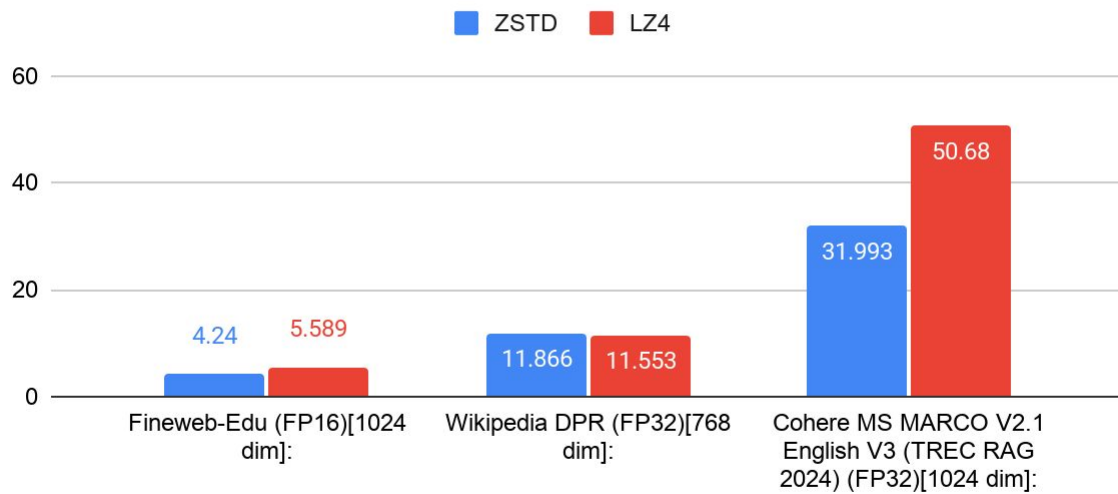
Metrics used:

- % Space Saving: Bitplane (vs Baseline)
- % Space Saving: No Bitplane (vs Baseline)
- Baseline: FP32, uncompressed

Bitplane vs No Bitplane % Improvement: Precomputed Results: (FP16 & FP32)

MS MARCO shows the most promising results here, with Wikipedia DPR and Fineweb-Edu showing tangible, but much lower numbers.

Not Quantized: % Space Saving Bitplane Compression vs Direct Compression



Precomputed Vector Embeddings:

- % Space savings = $(1 - (a/b)) * 100$
- Where:
 - a = Bitplane Compressed Size
 - b = Non-bitplane Compressed Size
 - Result rounded to the 3rd decimal point

Computed Results: (FP16 & FP32 & BF16)

Bitplaning shows compression improvements in all the embedding models when using ZSTD and LZ4.

Harrier OSS V1 has considerably higher compressibility than the other embedding models.

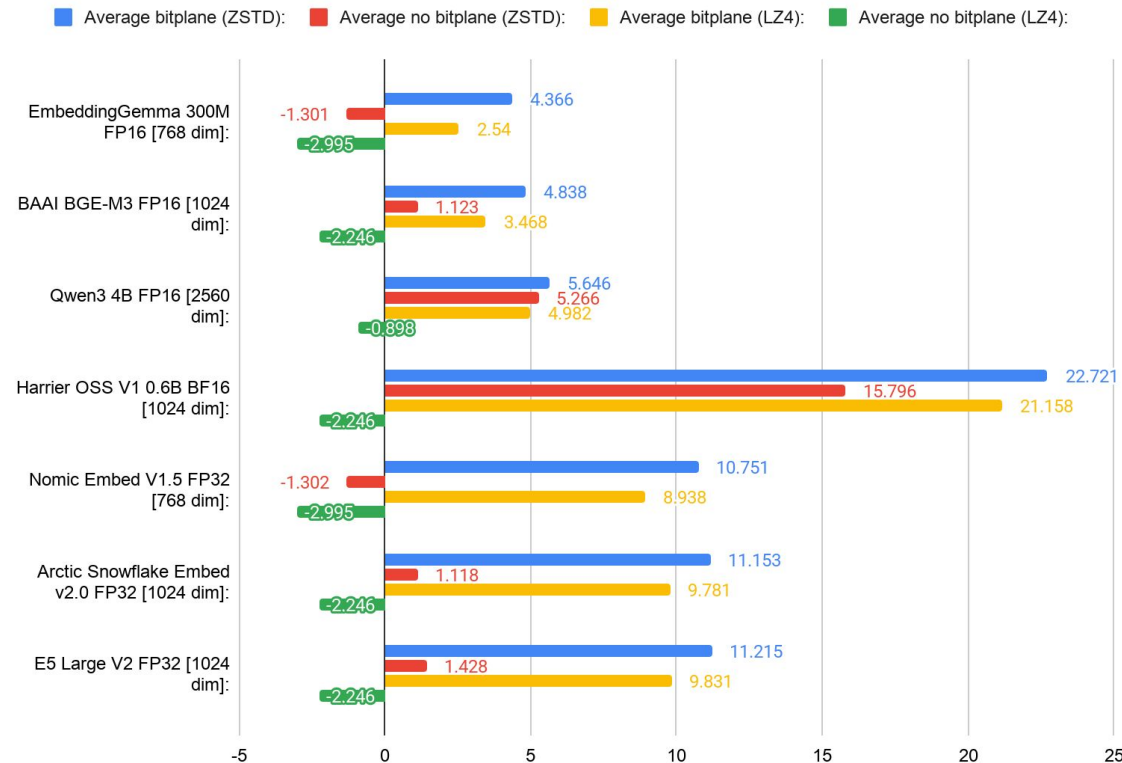
FP16: Typically ~4-5% reduction

BF16: High compression, maybe due to higher range: ~22% reduction

FP32: ~10-11% reduction

LZ4 with no bitplaning seems to fail, giving negative compression.

Computed Unquantized (AVG 9 Datasets): % Space Savings Bitplane and No Bitplane vs Baseline (ZSTD & LZ4)



Metrics used:

- % Space Saving: Bitplane (vs Baseline)
- % Space Saving: No Bitplane (vs Baseline)
- Baseline: FP32, uncompressed

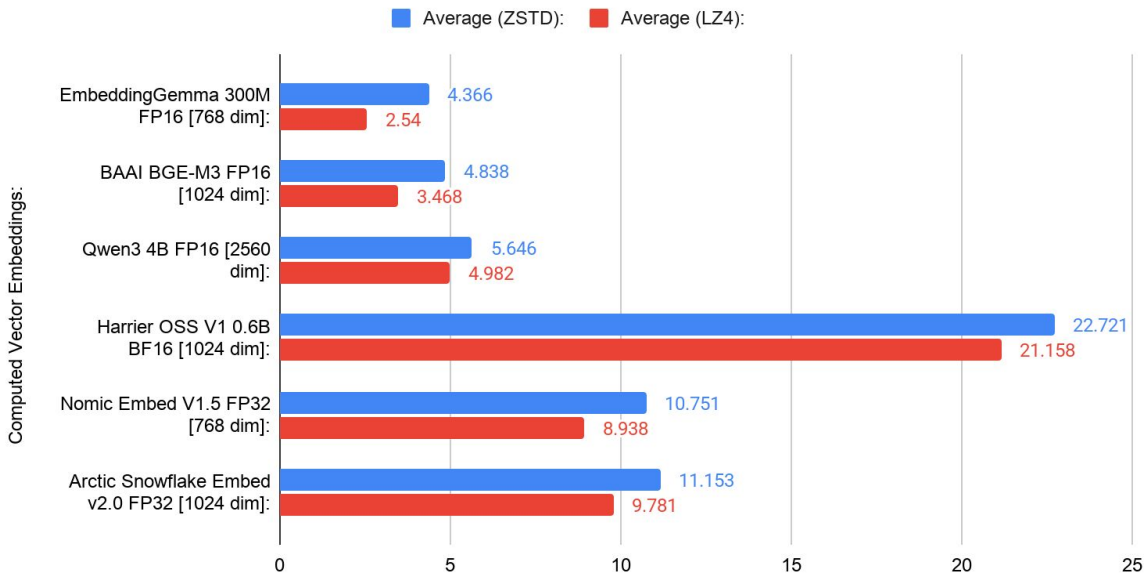
Bitplane vs No Bitplane %

Improvement: Computed Results: (FP16 & FP32 & BF16)

For ZSTD, bitplaning showed small improvements over no bitplaning, ranging from ~4-22% improvement.

LZ4 closely trails behind ZSTD in compressibility.

Computed: Average Across 9 Datasets of % Space Savings Unquantized Bitplane Compression vs Unquantized Compression (No bitplane)



- % Space savings = $(1 - (a/b)) * 100$
- Where:
 - a = Bitplane Compressed Size
 - b = Non-bitplane Compressed Size
 - Result rounded to the 3rd decimal point

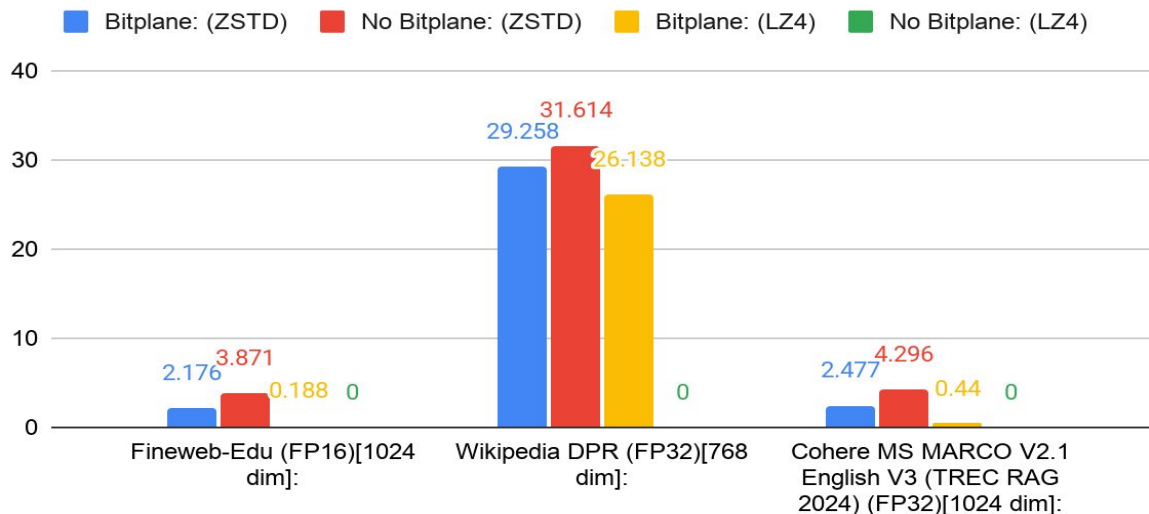
Precomputed Results: (INT8)

When it comes to ZSTD compression, bitplaning generally faired worse than no bitplaning, although they were fairedly close.

For LZ4 compression, bitplaning performs better than without bitplaning. It seems that LZ4 fails without bitplaning, giving 0% reduction in memory, while bitplaning exposes at least some redundancy for LZ4 to exploit.

Wikipedia DPR shows much better compression compared to the other two precomputed datasets for quantized compression.

Precomputed Quantized: % Space Savings Bitplane and No Bitplane vs Quantized Uncompressed (ZSTD & LZ4)



Metrics used:

- % Space Saving INT8 Bitplane vs Baseline
- % Space Saving INT8 Non-Bitplane vs Baseline

Baseline: INT8, Uncompressed

Given both are:

- Quantized
- Use same compression codec + block size
- Also include uncompressed scalar

Note: results were quantized from FP32 or FP16 to INT8 with symmetric scalar quantization

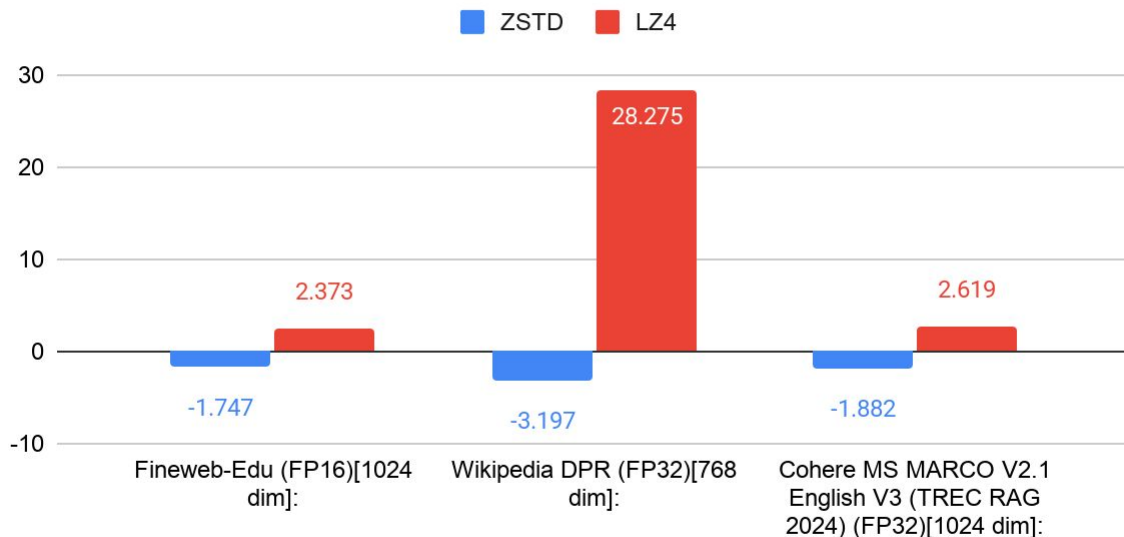
Bitplane vs No Bitplane % Improvement: Precomputed Results: (INT8)

Only Wikipedia DPR showed significant results using LZ4, around 28.275% reduction in memory compared to compression without bitplaning.

All of the ZSTD results show bitplaning worsens compression compared to without it.

The other LZ4 results show marginal improvements.

Quantized: % Space Savings bitplane vs no bitplane



Precomputed Vector Embeddings:

- % Space savings = $(1 - (a/b)) * 100$
- Where:
 - a = Bitplane Compressed Size (Quantized to INT8)
 - b = Non-bitplane Compressed Size (Quantized to INT8)
 - Result rounded to the 3rd decimal point

Computed Results: (INT8)

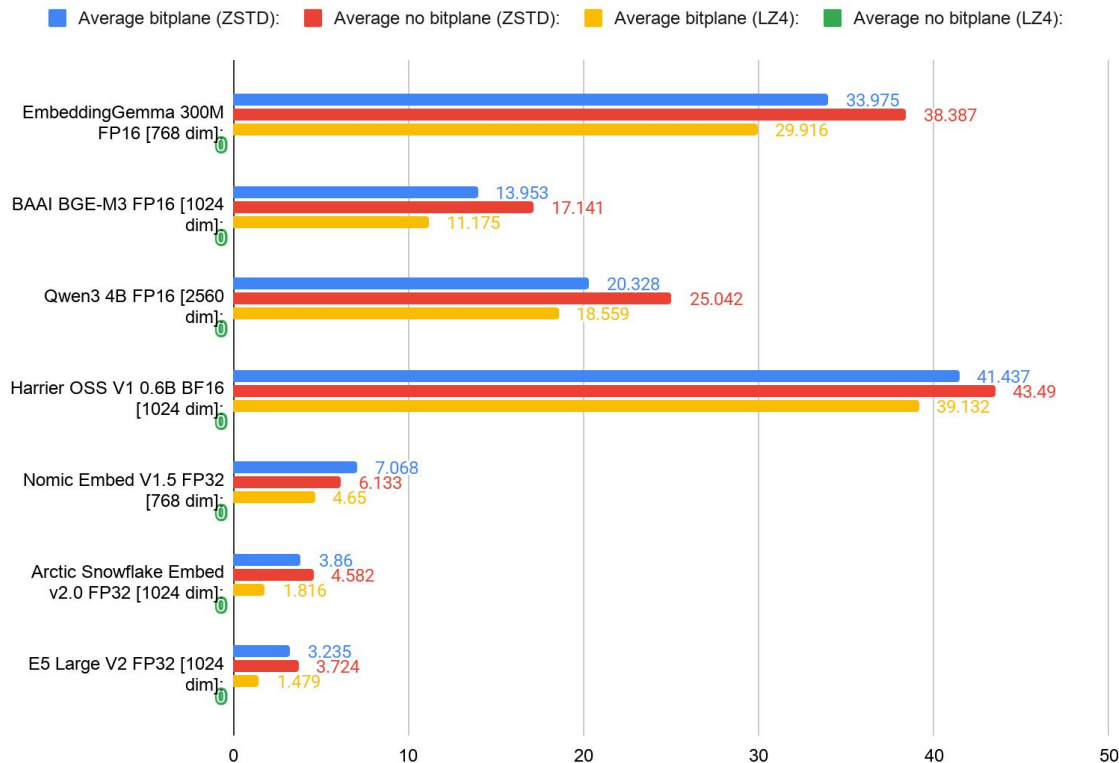
For ZSTD compression, bitplaning performs worse than without bitplaning across the board, with the exception of Nomic Embed. Though, bitplaning is fairly close to without bitplaning.

For LZ4 compression, bitplaning resulted in better compression than not bitplaning. It seems that the lower precision codecs have better compressibility.

Nomic embed is the only case where ZSTD with bitplaning compresses better than ZSTD without bitplaning

Note: results were quantized from FP32 or FP16 to INT8 with symmetric scalar quantization

Computed Quantized (AVG 9 Datasets): % Space Savings Bitplane and No Bitplane vs Quantized Uncompressed (ZSTD & LZ4)



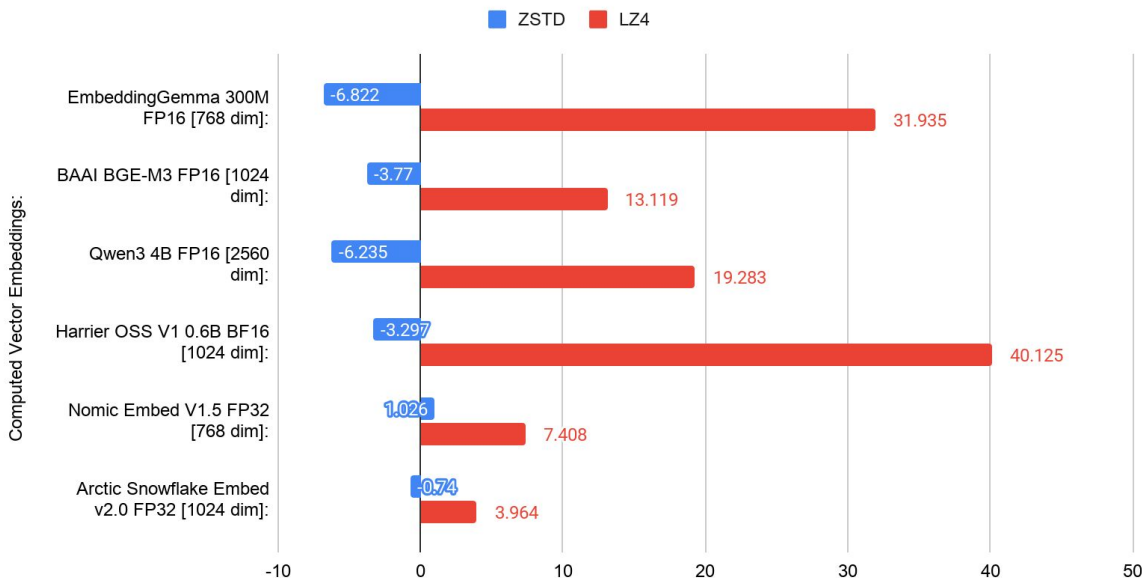
Same metrics as previous page

Bitplane vs No Bitplane % Improvement: Computed Results: (INT8)

Overall, results show that bitplaning is detrimental to compression ratios for ZSTD. For LZ4, bitplaning shows considerable improvements. The biggest outliers is LZ4 compression:

- Harrier OSS V1: ~40.125% reduction
- EmbeddingGemma: ~31.935% reduction

Computed: Average Across 9 Datasets of % Space Savings Bitplane Compression vs No Bitplane Compression (Quantized)



- % Space savings = $(1 - (a/b)) * 100$
- Where:
 - a = Bitplane Compressed Size (Quantized to INT8)
 - b = Non-bitplane Compressed Size (Quantized to INT8)
 - Result rounded to the 3rd decimal point

Detailed observations + conclusions

When Quantized: (Lossy Quantization + Lossless Bitplaning)

- Bitplaning makes ZSTD perform worse than without bitplaning
- Bitplaning greatly improves LZ4 compression compared to without bitplaning
 - Bitplaning exposes redundancy that the LZ4 codec cannot find normally (~0% reduction → ~30% reduction)

When Unquantized: (Lossless Bitplaning)

- Bitplaning performs better across the board for both ZSTD and LZ4. It yields great improvements over non-bitplaning.
 - Bitplaning exposes the redundancy within the sign and exponent bits as expected, however, there can also be some redundancy across the first few bits of the mantissa
 - Elastic Precision retrieval can potentially improve savings even further

Compression results varied not only between datasets, but between embedding model architectures:

- Best embedding models for Bitplaning: Harrier OSS (~40-45% reduction) and EmbeddingGemma (~30-37% reduction)
- Best Precomputed Results for Bitplaning:
 - Unquantized: Wikipedia DPR (~28% reduction)
 - Quantized: MS Marco (~30-50% reduction)

Thank you for your time

Any questions?

Next Steps in development:

- Add elastic retrieval (only utilizing the first few highly structured bitplanes)
- Add multimodal Embedding model support:
 - Image file support: jpg, png
 - Audio file support: mp3, wav, aac
 - Video file support? (maybe): mp4, mov?
- Add full Nvidia GPU support (and AMD gpu support if time persists)
- Re-add larger embedding models (if needed):
 - Qwen3 8B
 - Gemma 2 Embed 9B FP16 and FP32
- Maybe add more compression codecs? (Gzip, etc)

Explanation of discrepancies in results:

Why is ZSTD performing worse with bitplaning than without bitplaning for quantization?

- ZSTD models frequency distribution of bytes, and quantization already reduces the frequency distribution (makes it less spread out and more peaked). Bitplaning makes this worse by shredding the inter-value patterns that ZSTD can exploit, making it fail. It makes the frequency distribution more unpredictable.
- Bitplaning breaks inter-value patterns (correlation within a single byte)

Why does ZSTD perform better with bitplaning than without bitplaning for unquantized values?

- ZSTD struggles to find patterns in floats due to the sign, exponent, and mantissa bits being hard to compress. Putting it in a bitplane allows ZSTD to correlate the exponent bits and mantissa bits, which are now highly predictable in frequency, allowing for higher compression

Why is LZ4 giving 0% compression without bitplaning, but massively improves with bitplaning?

- LZ4 is a LZ77 byte matching algorithm. It relies on repeated patterns of exact byte matches. This almost never happens in pure floats or ints, but bitplaning creates these repeated identical byte patterns, allowing LZ4 to massively improve.
- When LZ4 cannot find repeated identical bytes, it returns no compression, hence the 0% memory reduction without bitplaning.
- The bitplane concatenation gives LZ4's sliding window more context for compression

Testing Methodology + Data Integrity

Computational Pipeline Summary:

Step 0: Vector Embedding Tensor Extraction (Precomputed & Computed)

Step 1: Normalization [normalizes to a range between -1 and 1, with a norm of 1]

Step 2: Quantization [Symmetric Scalar Quantization]

Step 3: Binary conversion

Step 4: Bitplane Disaggregation (per row) [includes tensor transformations]

Step 5: Bitpacking [+concatenation*]

Step 6: ZSTD & LZ4 Compression Analysis [+metrics]

*concatenation of bitplanes is needed so that ZSTD/LZ4 has enough context to compress effectively

Data Collection: Token Chunking Methodology

For each line:

1. Gets tokenized
2. Gets segmented into token chunks of (max token context of embedding model) tokens
3. Gets detokenized, and each detokenized chunk gets turned into a vector embedding row

Considerations for data integrity:

- For lines that are too long, it might hurt the semantic similarity if it cuts a sentence in half.
- If a line is too short, it is padded by default to fill 1 chunk, this may effect results.

Block Size for Compression Codecs: (ZSTD & LZ4)

For each embedding vector to be fed into the compressor:

1. Converted to bytes
2. Bytes are split into groups of size `<BLOCK_SIZE>`
3. Each block is compressed by the compressor
4. Compressed size for that vector = summation of all compressed sizes for that vector

$\% \text{ memory savings vs baseline} = (1 - \text{CompressedSize} / \text{OriginalSize}) * 100\%$

Note: Simulation runs were configured in such a way (block size, dimensions, etc) that every block is perfectly filled, no padding.

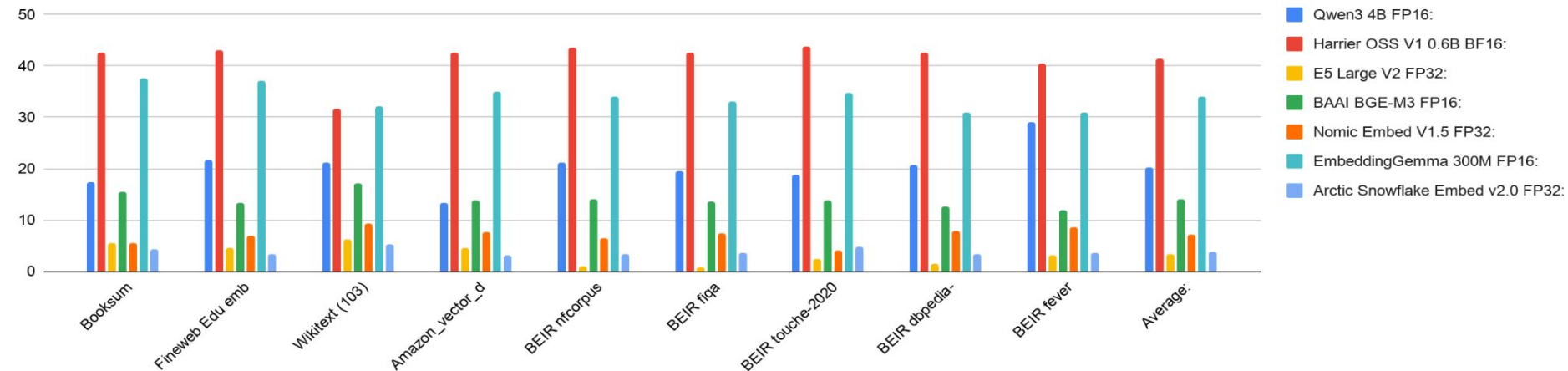
Some considerations that may skew results:

- Chunking
 - Potential internal padding (line too short)
 - Cut sentences break semantic similarity (line too long)
- Small windows for LZ4 + ZSTD
 - Small Dimensionality of models & datasets
 - Models with too small context may reset too often on datasets with too many tokens per line
 - Models with too large context may become underutilized, leading to inflated compression ratios

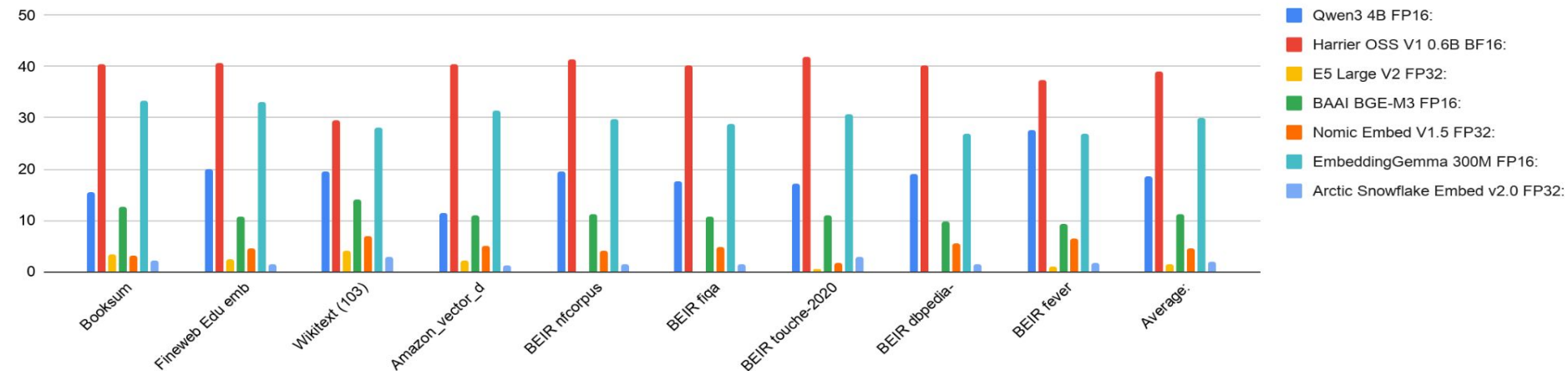
Many of these limitations are due to the constraints of random access limiting context to a single row, and the embedding models having dimensionality that is too small for ZSTD and LZ4.

Appendix

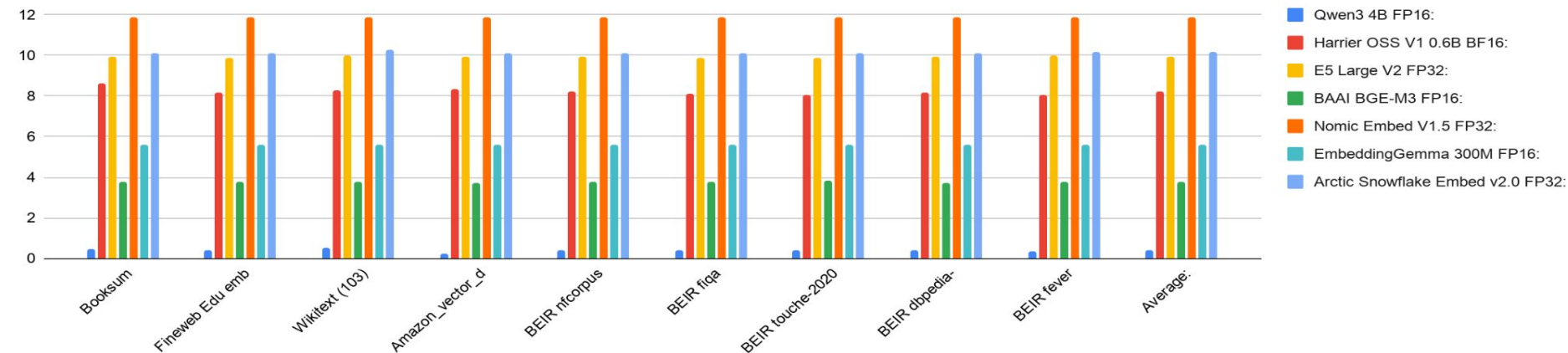
Computed: % Space Savings vs Quantization (ZSTD)



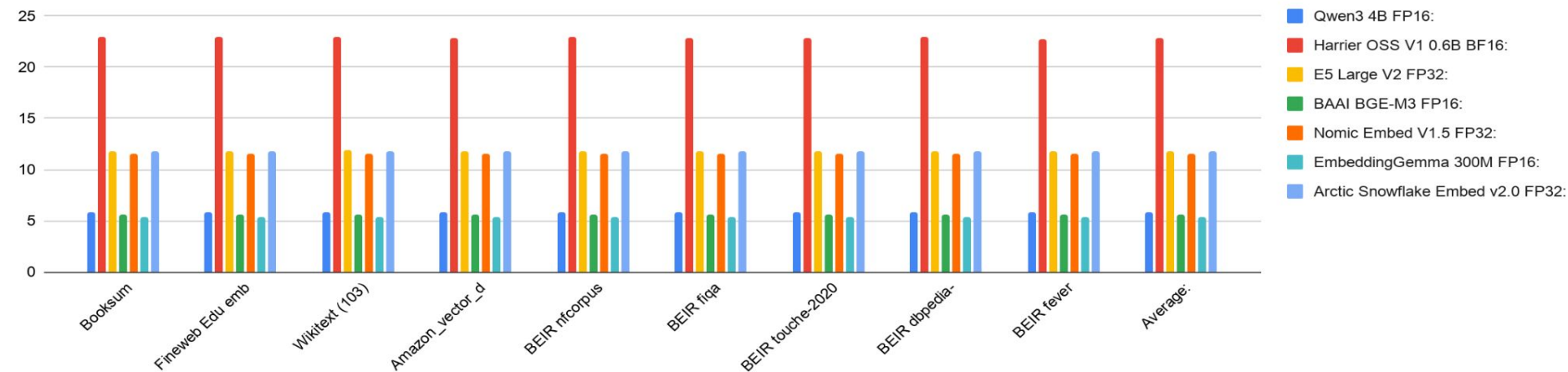
Computed: % Space Savings vs Quantization (LZ4)



Computed: % Space Saving: Unquantized Compressed vs Direct Compression (ZSTD)



Computed: % Space Saving: Unquantized Compressed vs Direct Compression (LZ4)



Precomputed Results:

Wikipedia showed the best overall compression results

Metrics Used:

- Lossless Savings:
 - Bitplane + Compression Quantized vs Quantized
- Total Savings:
 - Bitplane + Compression vs Baseline

Where:

- Baseline has no compression or quantization

Precomputed Dataset → INT8:	Lossless Savings (ZSTD):	Total Savings (ZSTD):
Fineweb-Edu (FP16)[1024 dim]:	2.176	50.897
WikipediaDPR (FP32)[768 dim]:	29.258	82.222
Cohere MSMARCO V2.1 English V3 (FP32)[1024 dim]:	2.477	75.524
Precomputed Dataset → INT8:	Lossless Savings (LZ4):	Total Savings (LZ4):
Fineweb-Edu (FP16)[1024 dim]:	0.188	49.899
WikipediaDPR (FP32)[768 dim]:	26.138	81.438
Cohere MSMARCO V2.1 English V3 (FP32)[1024 dim]:	0.44	75.013

Computed Results:

Overall, the 2 best models for compression shown are Harrier OSS V1 and EmbeddingGemma 300M. They both show significant improvements in compressibility, ~30-40% reduction in memory compared to quantization alone

Metrics Used:

- Lossless Savings:
 - Bitplane + Compression Quantized vs Quantized
- Total Savings:
 - Bitplane + Compression vs Baseline

Where:

- Baseline has no compression or quantization

Embedding model avg:	Lossless Savings (ZSTD):	Lossless Savings (LZ4):	Total Savings (ZSTD):	Total Savings (LZ4):
EmbeddingGemma 300M FP16 [768 dim]:	33.975	29.916	66.815	64.775
BAAI BGE-M3 FP16 [1024 dim]:	13.953	11.175	56.808	55.414
Qwen3 4B FP16 [2560 dim]:	20.328	18.559	60.101	59.216
Harrier OSS V1 0.6B BF16 [1024 dim]:	41.437	39.132	70.604	69.447
Nomic Embed V1.5 FP32 [768 dim]:	7.068	4.65	76.646	76.038
Arctic Snowflake Embed v2.0 FP32 [1024 dim]:	3.86	1.816	75.871	75.358
E5 Large V2 FP32 [1024 dim]:	3.235	1.479	75.714	75.273